

Object-Oriented Programming in Java

Emily Lucia Antosch

18.09.2026

Today's route

1. Introduction	2
2. Abstract Classes and Methods	4
3. Interfaces	15
4. Comparison and Sorting	27
5. License Notice	39

1. Introduction

From Inheritance to Contracts

Last lecture

Today

Inheritance

Interfaces

Comparable

Reuse and override

Model shared contracts

Order and sort objects

2. Abstract Classes and Methods

What Is a Plain Shape?

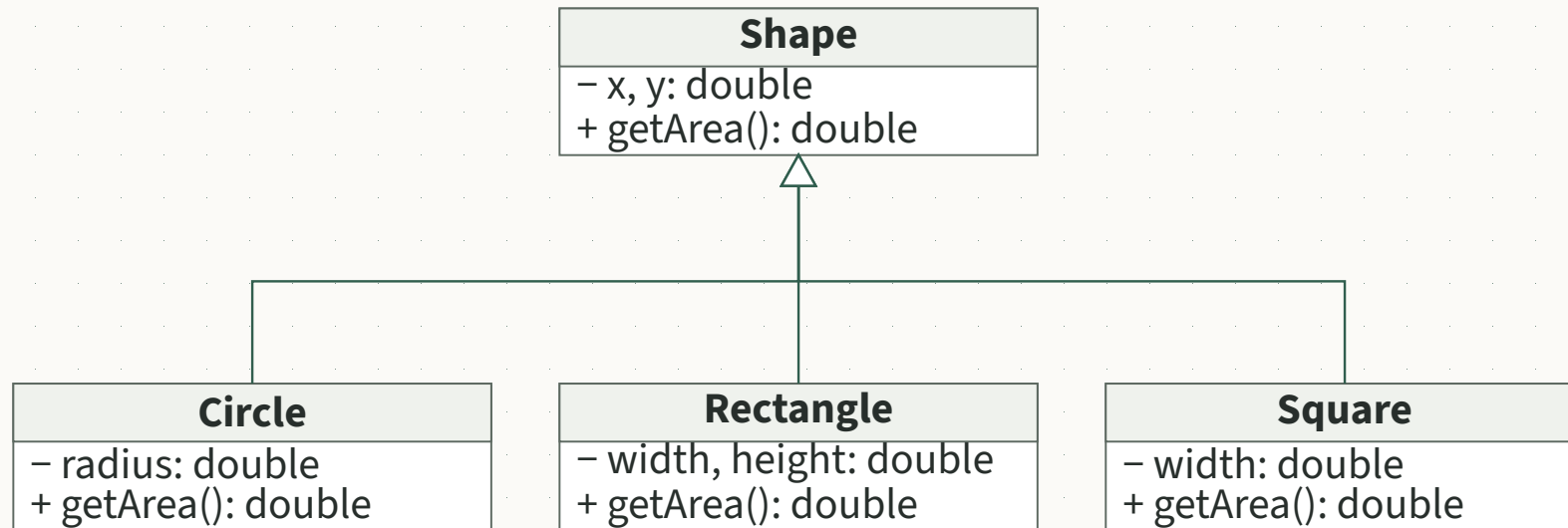


Should we create a Shape object? What would its area be?

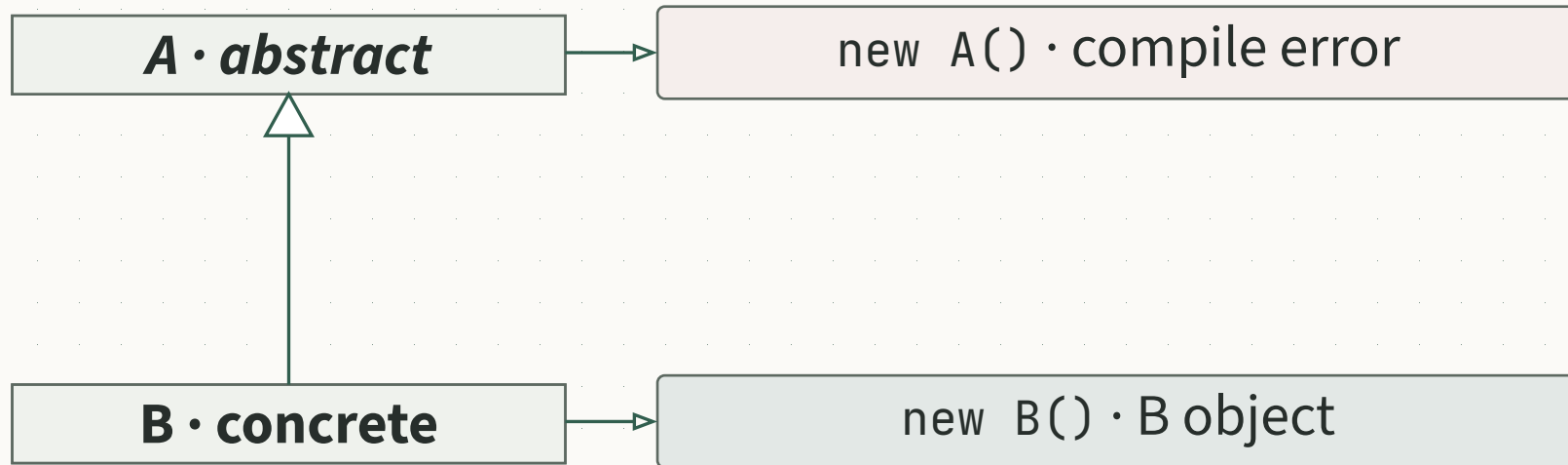
What Is a Plain Shape?



Should we create a Shape object? What would its area be?



Abstract Classes Cannot Be Instantiated



`A ref = new B();` is valid

Abstract Classes Cannot Be Instantiated

```
1 public abstract class A {  
2     // Shared state and behavior  
3 }  
4 public class B extends A {  
5     // Concrete subclass  
6 }
```

Java

Inside main():

```
1 A invalid = new A(); // Compile error: abstract class  
2 A valid = new B();   // Concrete object, abstract reference type
```

Java

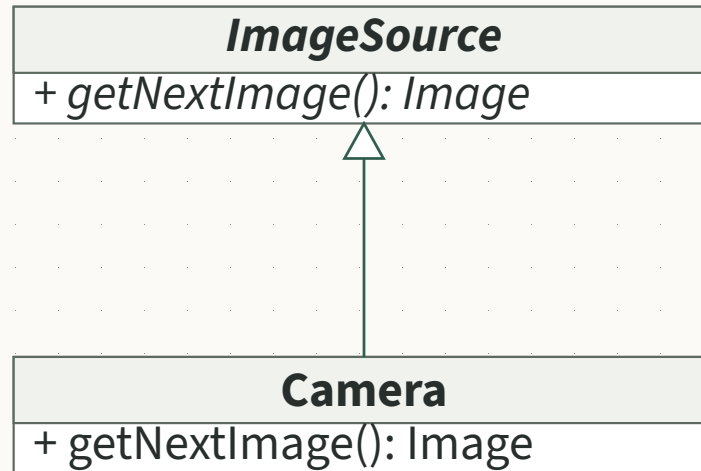
Abstract Methods Declare a Contract

```
1 public abstract class ImageSource {  
2     String name;  
3     public abstract Image getNextImage();  
4 }
```

Java

No method body: a concrete subclass must supply it.

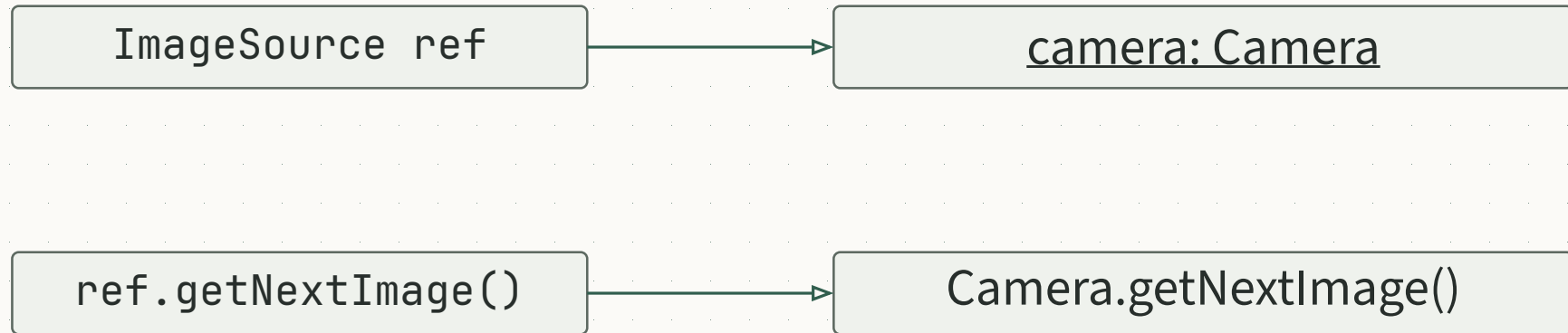
Abstract Methods Declare a Contract



Declaration above · implementation below

Italics mark abstract classes and methods in UML.

Calling Through an Abstract Type

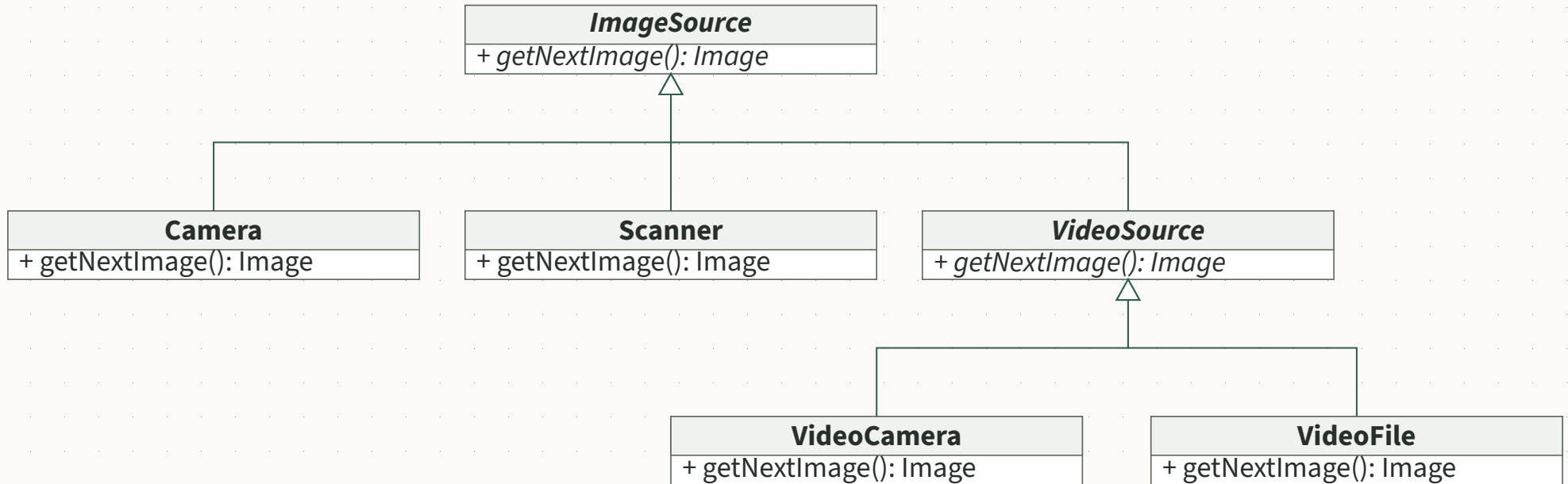


Abstract reference type · concrete implementation runs

```
1 ImageSource ref = new Camera();  
2 Image image = ref.getNextImage();
```

Java

A Subclass Can Stay Abstract



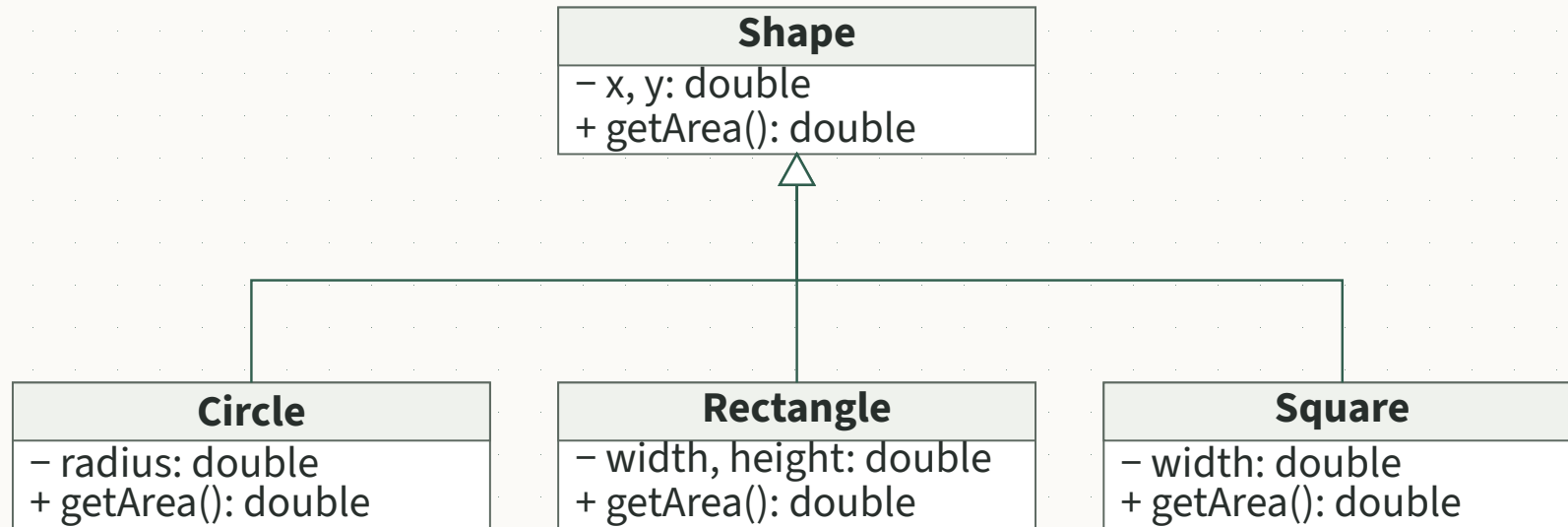
Still missing an implementation? The subclass must remain abstract.

Task: Make Shape Abstract

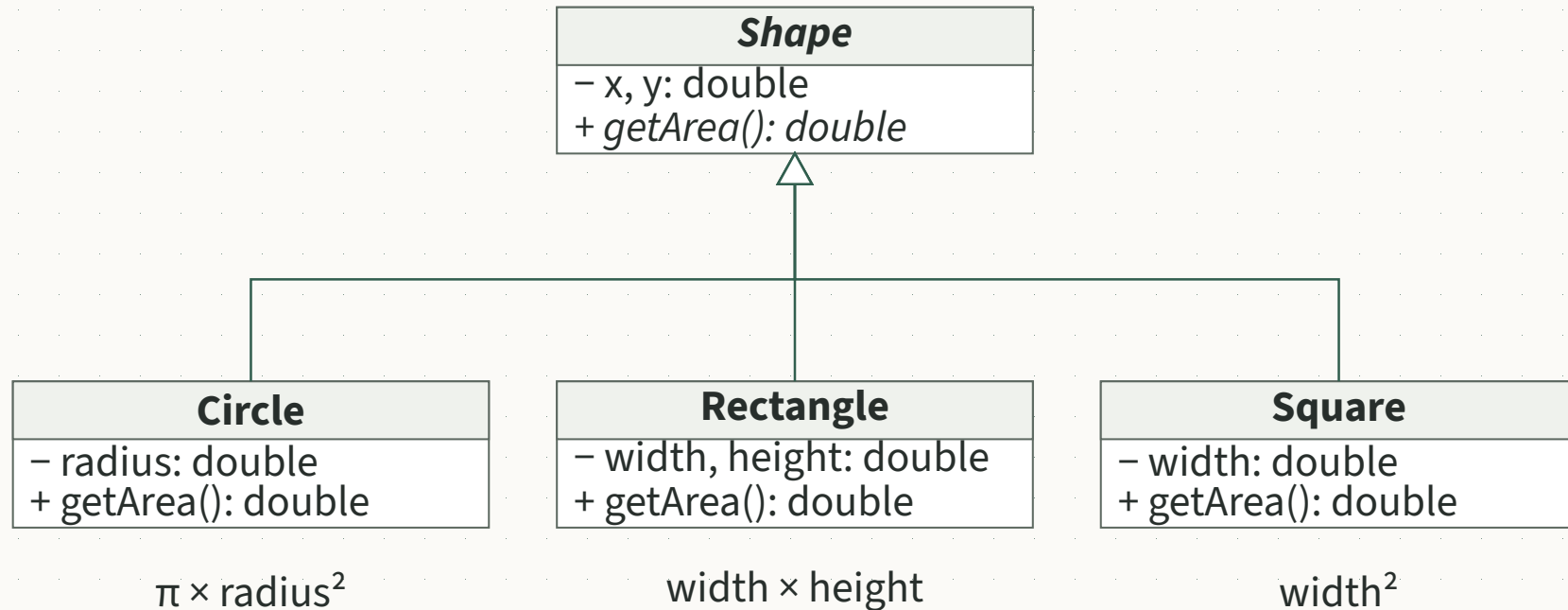


Task 1

Prevent plain Shape objects. Require each concrete shape to implement `getArea()`.



Task: Make Shape Abstract



Italics = abstract · instantiate only the concrete shapes

Task: Make Shape Abstract

```
1 public abstract class Shape {  
2     // Existing fields and constructor  
3     public abstract double getArea();  
4 }
```

Java

In Circle:

```
1 @Override  
2 public double getArea() {  
3     return Math.PI * radius * radius;  
4 }
```

Java

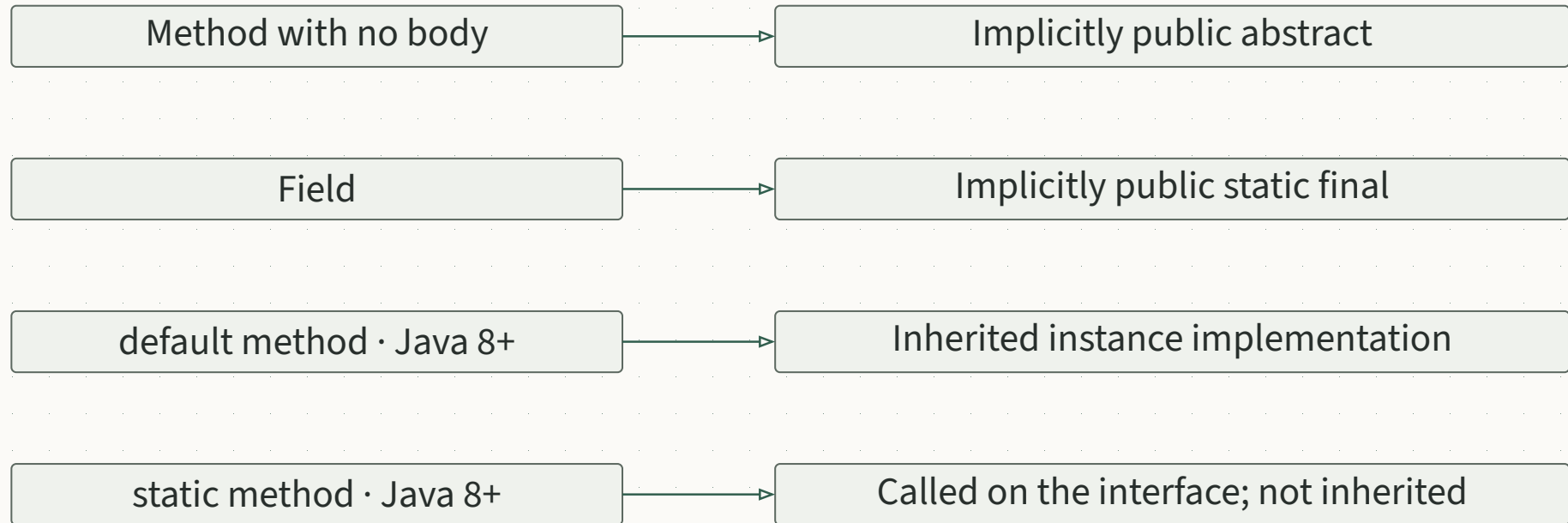
3. Interfaces

Classes and Interfaces

Concrete class	<i>Abstract class</i>	«interface» Interface
Instance + static fields Implemented methods Constructors Can instantiate	Instance + static fields Implemented + abstract methods Constructors Cannot instantiate	Constants; no instance fields Abstract + default/static methods No constructors Cannot instantiate

A concrete class must implement all inherited abstract methods.

What Can an Interface Declare?

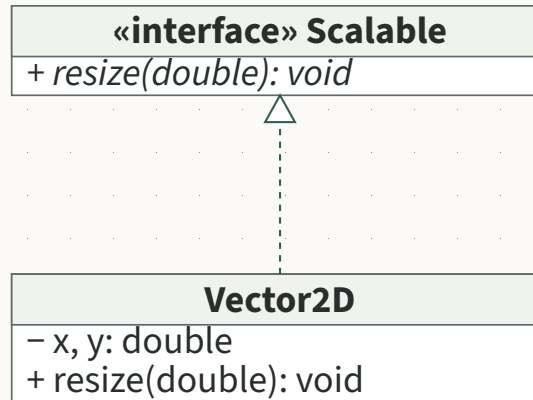


Java 9+ also permits private helper methods.

Declare a Shared Capability

```
1 public interface Scalable {  
2     void resize(double factor);  
3 }
```

Java



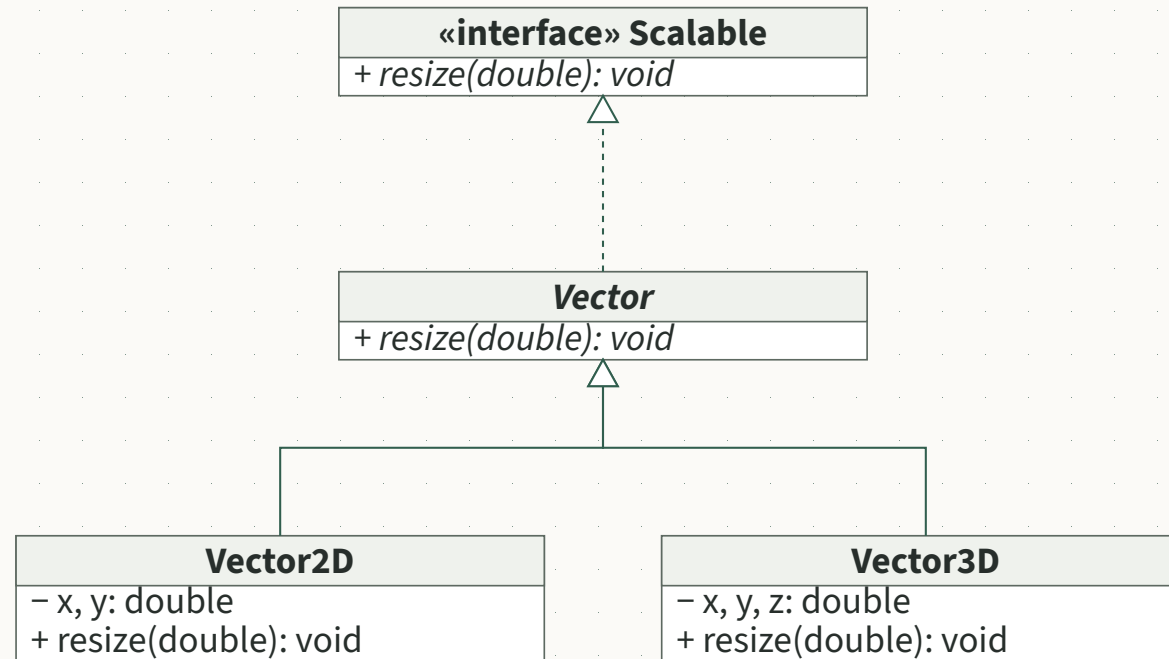
Dashed triangle = implements

Implement the Contract

```
1  public class Vector2D implements Scalable {  
2      private double x, y;  
3      public Vector2D(double x, double y) {  
4          this.x = x;  
5          this.y = y;  
6      }  
7      @Override  
8      public void resize(double factor) {  
9          x *= factor;  
10         y *= factor;  
11     }  
12     public double getX() { return x; }  
13     public double getY() { return y; }  
14 }
```

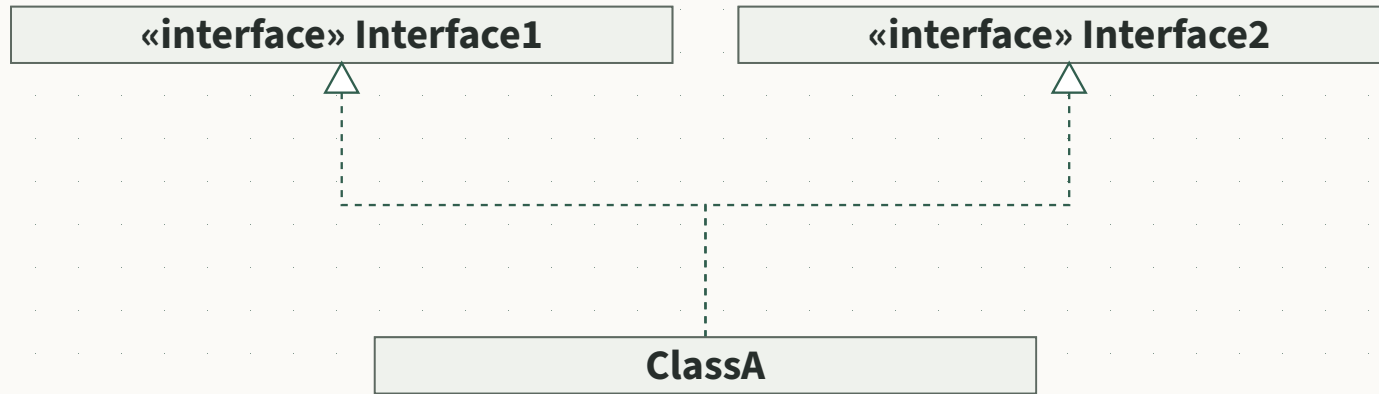
Java

Defer Implementation to Subclasses



Concrete subclasses implement `resize()`.

Implement Multiple Interfaces

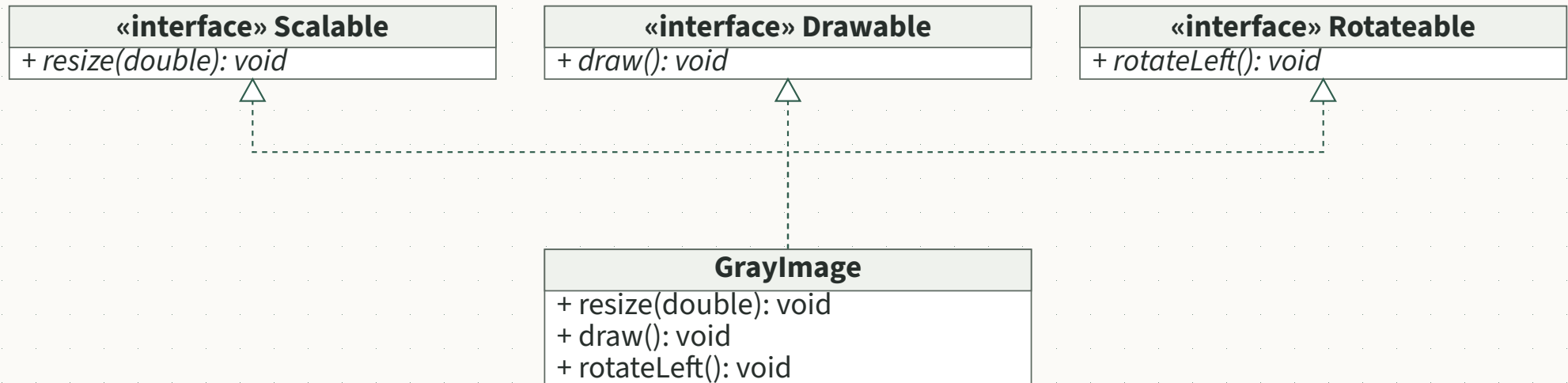


One superclass · any number of interfaces

```
1 class ClassA implements Interface1, Interface2 {  
2     // Implement both contracts  
3 }
```

Java

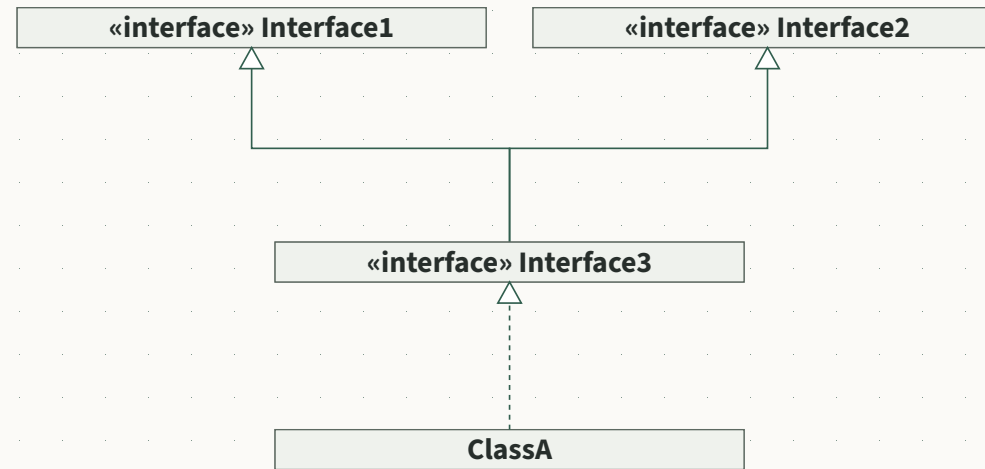
Implement Multiple Interfaces



```
1 public class GrayImage implements Scalable, Drawable, Rotateable {
2     // Implement resize(), draw(), and rotateLeft()
3 }
```

Java

Interfaces Can Extend Multiple Interfaces

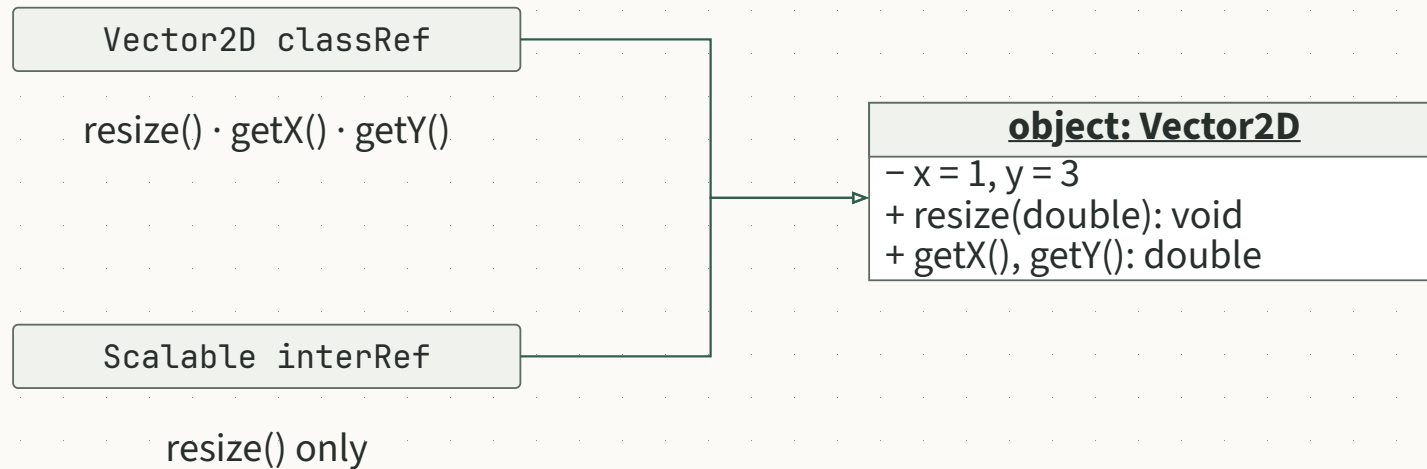


Interfaces extend interfaces · classes implement interfaces

```
1 interface Interface3 extends Interface1, Interface2 { }
2 class ClassA implements Interface3 {
3     // Implement all inherited abstract methods
4 }
```

Java

One Object, Two Reference Types



One object · the reference type determines available methods

```
1 Vector2D classRef = new Vector2D(1, 3);  
2 Scalable interRef = classRef;
```

Java

One Object, Two Reference Types



Which calls compile?

```
1 Vector2D classRef = new Vector2D(1, 3);
2 Scalable interRef = classRef;
3
4 classRef.resize(1.5);
5 System.out.println(classRef.getX());
6 interRef.resize(1.5);
7 System.out.println(interRef.getX());
```

Java

One Object, Two Reference Types



Which calls compile?

```
1 Vector2D classRef = new Vector2D(1, 3);  
2 Scalable interRef = classRef;  
3  
4 classRef.resize(1.5);  
5 System.out.println(classRef.getX());  
6 interRef.resize(1.5);  
7 System.out.println(interRef.getX());
```

Java

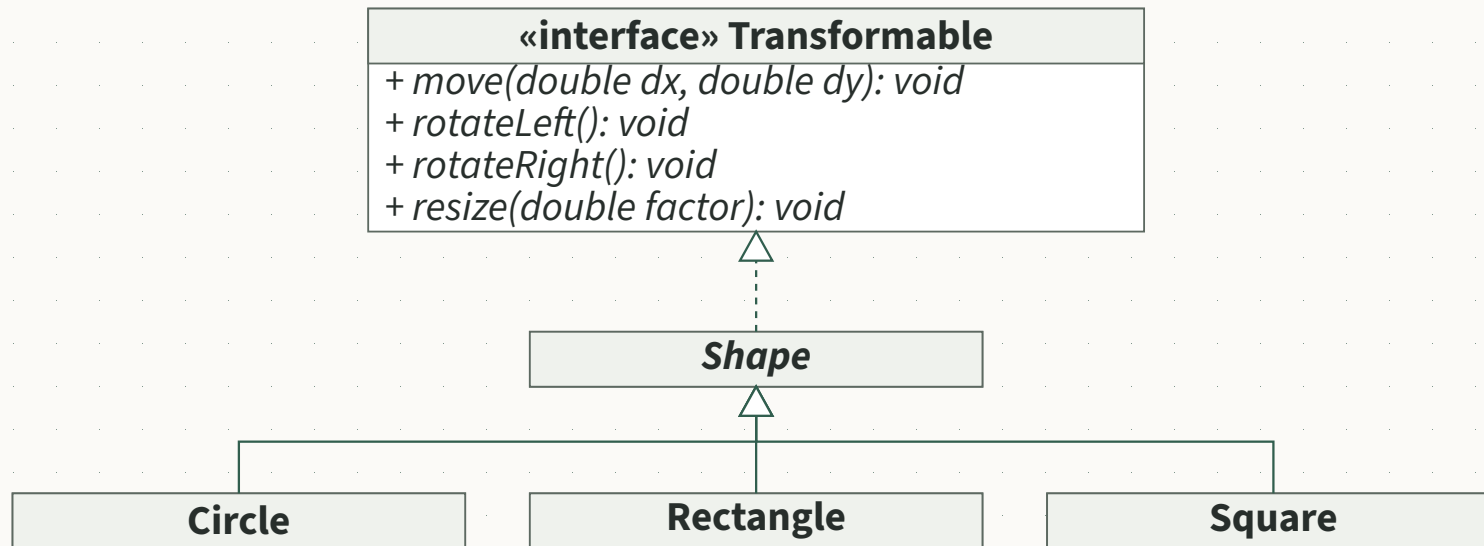
Only `interRef.getX()` fails: `Scalable` does not declare it.

Task: Transform Geometric Shapes



Task 2

Define Transformable. Support moving, 90° left/right rotation, and scaling in every concrete shape.

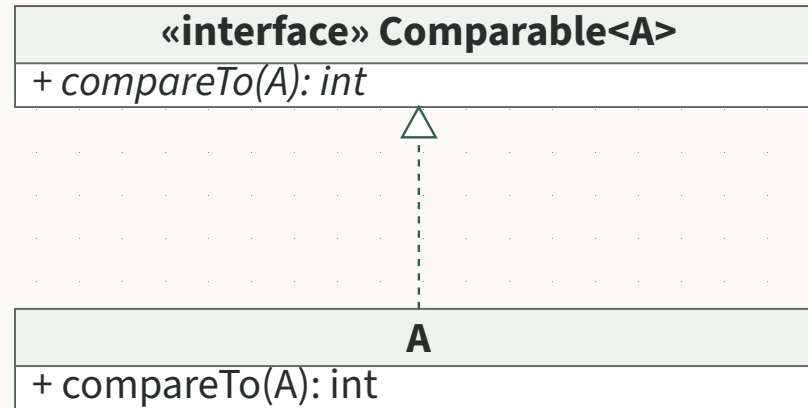


4. Comparison and Sorting

Comparable Defines the Natural Order

```
1 public interface Comparable<T> {  
2     int compareTo(T other);  
3 }
```

Java



Replace the type parameter with your class.

Read the Sign, Not the Exact Value

`this.compareTo(other)` returns an int

< 0

0

> 0



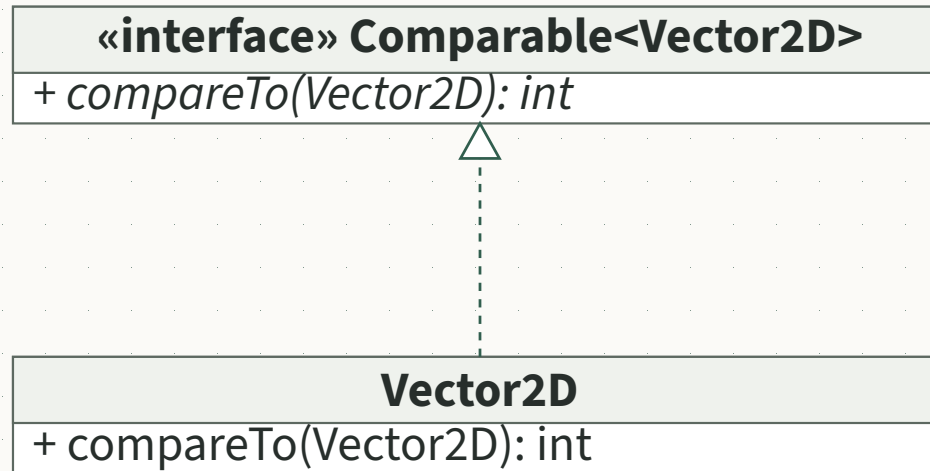
this comes before other

Same position in the ordering

this comes after other

Only the sign matters · zero does not mean object identity

Compare Vectors by Magnitude



Replace the type parameter with your class.

Keep Scalable too: `implements Scalable, Comparable<Vector2D>`.

Compare Vectors by Magnitude

Add to Vector2D:

```
1 public double getAbs() {  
2     return Math.sqrt(x * x + y * y);  
3 }  
4 @Override  
5 public int compareTo(Vector2D other) {  
6     return Double.compare(getAbs(), other.getAbs());  
7 }
```

Java

`Double.compare` returns a negative value, zero, or a positive value.

Sort a List of Vectors

```
1 // Import java.util.ArrayList and java.util.Collections.
2 // Inside main():
3 ArrayList<Vector2D> vectors = new ArrayList<>();
4 vectors.add(new Vector2D(0, 5));
5 vectors.add(new Vector2D(0, -1));
6 vectors.add(new Vector2D(7, 8));
7 vectors.add(new Vector2D(0, 0));
8 Collections.sort(vectors);
9 for (Vector2D vector : vectors) {
10     System.out.println(vector.getAbs());
11 }
```

Java

Sort a List of Vectors



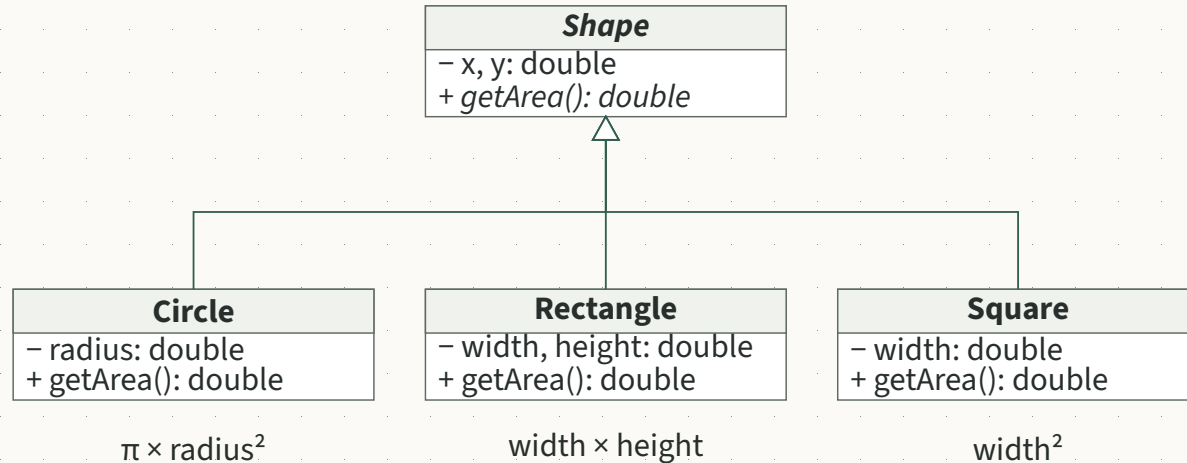
Ascending natural order · the list is changed in place

Task: Compare Shapes by Area



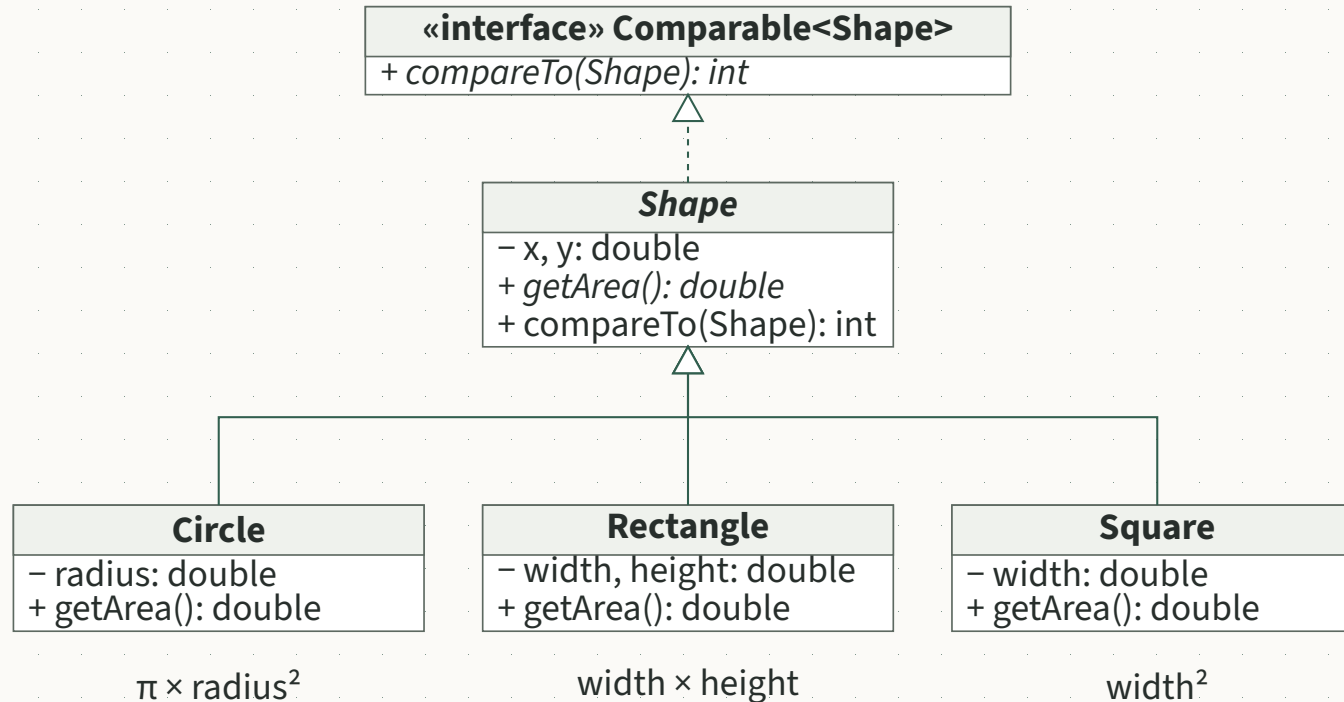
Task 3

Implement Comparable. Order shapes by area. Where should compareTo() go?



Italics = abstract · instantiate only the concrete shapes

Task: Compare Shapes by Area



Subclasses inherit compareTo(); each supplies its own getArea().

Task: Compare Shapes by Area

```
1 public abstract class Shape implements Comparable<Shape> {
2     // Existing fields, constructor, and other methods
3     public abstract double getArea();
4
5     @Override
6     public int compareTo(Shape other) {
7         return Double.compare(getArea(), other.getArea());
8     }
9 }
```

Java

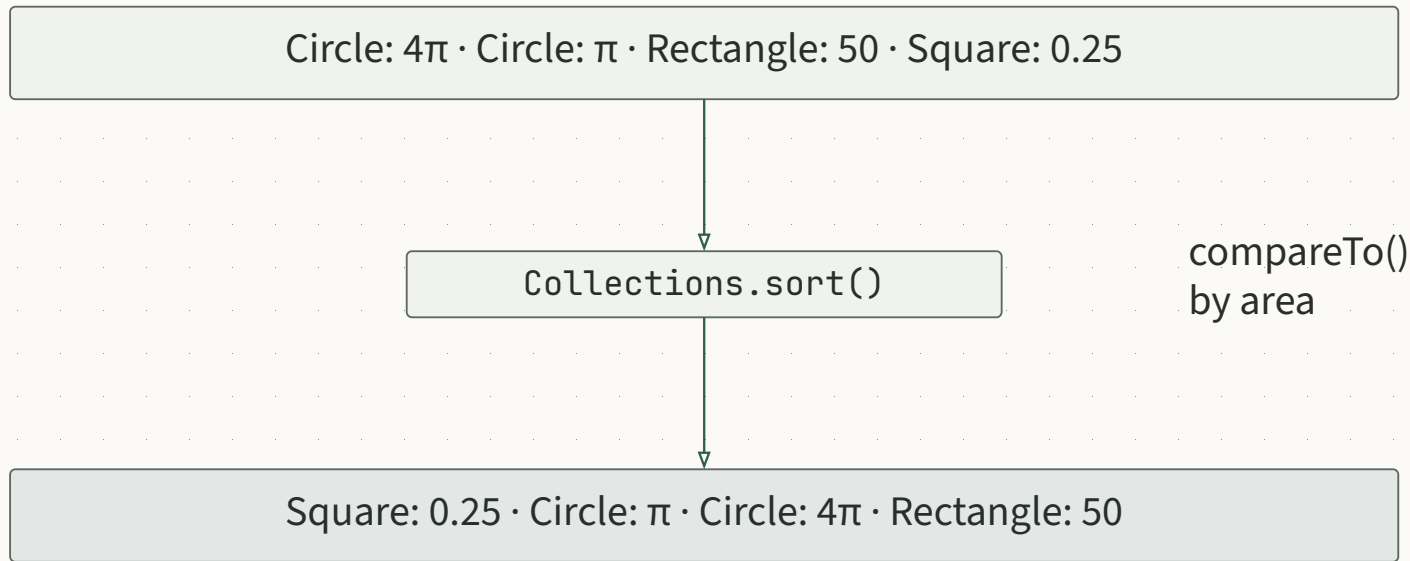
`getArea()` dispatches to each concrete shape's implementation.

Sort Different Shapes Together

```
1  // Import java.util.ArrayList and java.util.Collections.
2  // Inside main(); use the constructors from lecture 5:
3  ArrayList<Shape> shapes = new ArrayList<>();
4  shapes.add(new Circle(0.0, 0.0, 2.0));
5  shapes.add(new Circle(0.0, 0.0, 1.0));
6  shapes.add(new Rectangle(0.0, 0.0, 10.0, 5.0));
7  shapes.add(new Square(0.0, 0.0, 0.5));
8  Collections.sort(shapes);
9  for (Shape shape : shapes) {
10     System.out.println(shape.getArea());
11 }
```

Java

Sort Different Shapes Together



Ascending natural order · the list is changed in place

5. License Notice

Attribution

- Shared under CC BY-NC-SA 4.0 and the applicable public license.
- Based on teaching material by Prof. Dr. Marc Hensel.
- Text, diagrams, and layout adapted. Original material supplied privately; no public source link is available.